

Assignment 4 – End-to-End Testing and Application Containerization

Course: CISC/CMPE-327 – Software Quality Assurance

Total Marks: 10 | **Due Date:** 30 November 2025

Assignment Overview

This assignment extends the *Library Management System* project from previous labs and assignments. It introduces **browser-based end-to-end (E2E) testing** and **application containerization**.

In the first part, you will design and execute automated E2E tests that simulate real user behavior on your Flask-based web application (e.g., adding books, viewing records).

In the second part, you will containerize your application using **Docker** to ensure it runs consistently across environments.

Finally, you will deploy your Docker image to **Docker Hub** to demonstrate reproducibility and delivery readiness.

Getting Started: You can watch this [Playwright beginner tutorial](#) and visit the [Playwright Python documentation](#) to get started.

Tasks

Task 1 – Browser-Based E2E Testing (4 marks)

Write automated browser-based E2E tests for your Flask application.

You may use **Playwright (Python)** or **Selenium (Python)** – other testing types (HTTP-based, CI-based) exist but are *not required* for this assignment.

Requirements

- The E2E test must launch a real browser session (headless or non-headless).
- Automate **at least two** realistic user flow, such as:
 - i. Add a new book to the catalog (fill title, author, ISBN, copies)
 - ii. Verify the book appears in the catalog
 - iii. Navigate to borrow book page
 - iv. Borrow the book using a patron ID
 - v. Verify the borrow confirmation message appears
- Include assertions verifying that expected UI elements/text appear.
- Tests must run automatically using a command (e.g., `pytest tests/test_e2e.py`).

Task 2 – Application Containerization (3 marks)

Containerize your Flask application using **Docker** so that it runs on port 5000. Your container must use the internal SQLite database already included in your project. No external database or volume mounting is required.

Requirements

- Create a Dockerfile at the project root.
- The image must install all dependencies, copy source files, expose port 5000, and run the Flask server.
- After building, the app should be accessible at <http://localhost:5000>.

Example Commands

```
docker build -t library-app .  
docker run -p 5000:5000 library-app
```

Task 3 – Docker Hub Deployment (2 marks)

Demonstrate your understanding of image deployment and reproducibility.

Requirements

- Create a Docker Hub account.
- Tag your image and push it to Docker Hub:

```
docker tag library-app yourdockerhubusername/library-app:v1  
docker push yourdockerhubusername/library-app:v1
```

- Delete the local image and re-pull it from Docker Hub:

```
docker rmi yourdockerhubusername/library-app:v1  
docker pull yourdockerhubusername/library-app:v1  
docker run -p 5000:5000 yourdockerhubusername/library-app:v1
```

- Include screenshots of successful push, deletion, pull, and run in your report.
-

What to Submit

1. Code Files

Push your full project (with Flask code, E2E tests, and Dockerfile) to your GitHub repository.

In addition to your existing code from previous assignments, ensure the following structure:

```
/tests
├─ test_e2e.py
Dockerfile
requirements.txt
```

2. Report (PDF) – A4_LastName_Last4Digit_StudentID.pdf

Include the following sections:

1. **Student Information** (name, ID, date)
2. **E2E Testing Approach** (tool used, tested features, assertions)
3. **Execution Instructions** (clear commands to run tests and container)
4. **Test Case Summary** (table of actions, expected results)
5. **Dockerization Process** (steps + screenshots of build and run)
6. **Docker Hub Deployment** (push/pull proof with screenshots)
7. **Challenges and Reflections** (brief paragraph on difficulties and learnings)

Mark Distribution

Component	Marks	Details
Browser-Based E2E Testing	4	Functional test script verifying key user flows (add/search book, etc.)
Dockerization	3	App container builds and runs successfully on port 5000
Docker Hub Deployment	2	Image pushed, deleted locally, re-pulled, and verified running
Documentation & Submission Quality	1	Clear PDF report with instructions and screenshots
Total	10	

Important Guidelines

- Use only **Playwright** or **Selenium** for browser-based testing.
 - Keep your E2E test simple and focused on user behavior (avoid backend unit tests here).
 - The Dockerfile must use a Python base image (e.g., python:3.11-slim).
 - The Flask app must automatically initialize its SQLite database on startup.
 - Expose port 5000 and ensure the app runs using: flask run --host=0.0.0.0.
 - Do not use external databases or volume mounting in this assignment.
 - Screenshots must be clear and legible in the report.
 - Ensure your Docker image size is reasonable (< 500 MB if possible).
-

Submission Steps

Step 1 – Push to GitHub:

Push your complete project to GitHub, including all source code, E2E tests in tests/, Dockerfile. Ensure your repository is public or accessible to the instructor.

Step 2 – Submit to OnQ:

Submit your GitHub repository URL and a PDF report named A4_LastName_Last4Digit_StudentID.pdf containing all required commands and descriptions with clear screenshots.